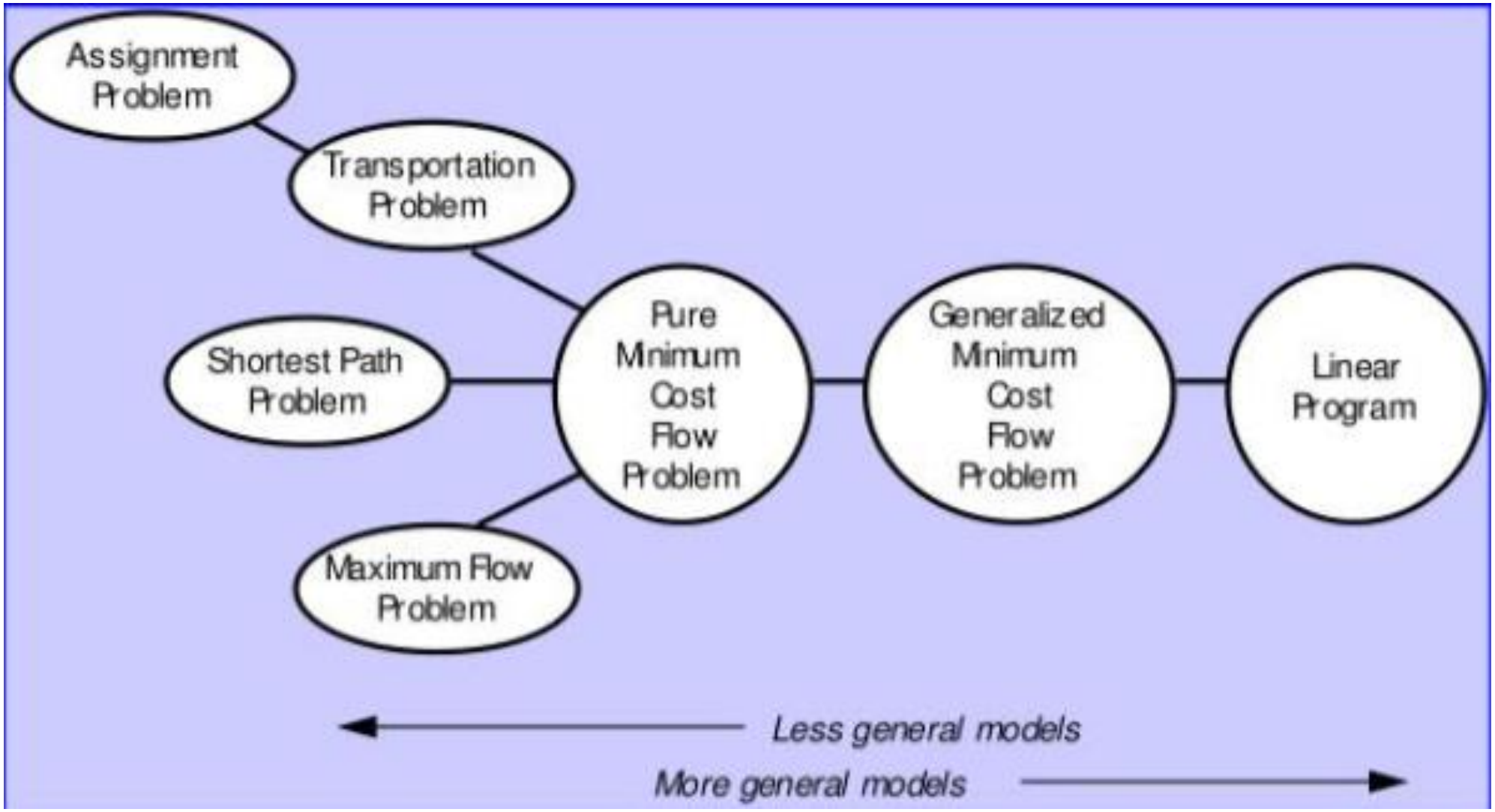
OR 6205: Deterministic Operations Research
Spring 2026

Deterministic Operations Research

Class 24: Network Optimization

Instructor: Priscila de Azevedo Drummond
deazevedodrummond.p@northeastern.edu

Comparison between different Network Models



Difference between models

Minimize $\sum_{i=1}^{m}\sum_{j=1}^{n} c_{ij}x_{ij}$

Subject to:

$$\sum_{j=1}^{n} x_{ij} \leq a_i \quad \text{for } i = 1, 2, \ldots, m$$

$$\sum_{i=1}^{m} x_{ij} \geq b_j \quad \text{for } j = 1, 2, \ldots, n$$

$$x_{ij} \geq 0 \quad \text{for } i = 1, 2, \ldots, m \text{ and } j = 1, 2, \ldots, n.$$

The goal is to minimize $\sum_{i=1}^{m}\sum_{j=1}^{n} t_{i,j}x_{i,j}$ subject to:

- $x_{r,s} \geq 0; \forall r = 1 \ldots m, s = 1 \ldots n$
- $\sum_{s=1}^{m+n} x_{i,s} - \sum_{r=1}^{m+n} x_{r,i} = a_i; \forall i = 1 \ldots m$
- $\sum_{r=1}^{m+n} x_{r,m+j} - \sum_{s=1}^{m+n} x_{m+j,s} = b_{m+j}; \forall j = 1 \ldots n$
- $\sum_{i=1}^{m} a_i = \sum_{j=1}^{n} b_{m+j}$

Min Cost Flow

Minimize $z = \sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij} x_{ij}$

Subject to

$\sum_{j=1}^{n} x_{ij} - \sum_{k=1}^{n} x_{ki} = b_i$ for each node i=1, ..., n -> (Flow balance, or flow conservation or Kirchhoff equations)

$l_{ij} \leq x_{ij} \leq u_{ij}$ for each arc (i, j) - (Bound inequalities)

Shortest Path

$$\min z = \sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij} x_{ij}$$

Subject to:

$$\sum_{j=1}^{n} x_{ij} - \sum_{k=1}^{n} x_{ki} = \begin{cases} 1 & \text{if } i = s \text{ (source)} \\ 0 & \text{otherwise} \\ -1 & \text{if } i = t \text{ (sink)} \end{cases}$$

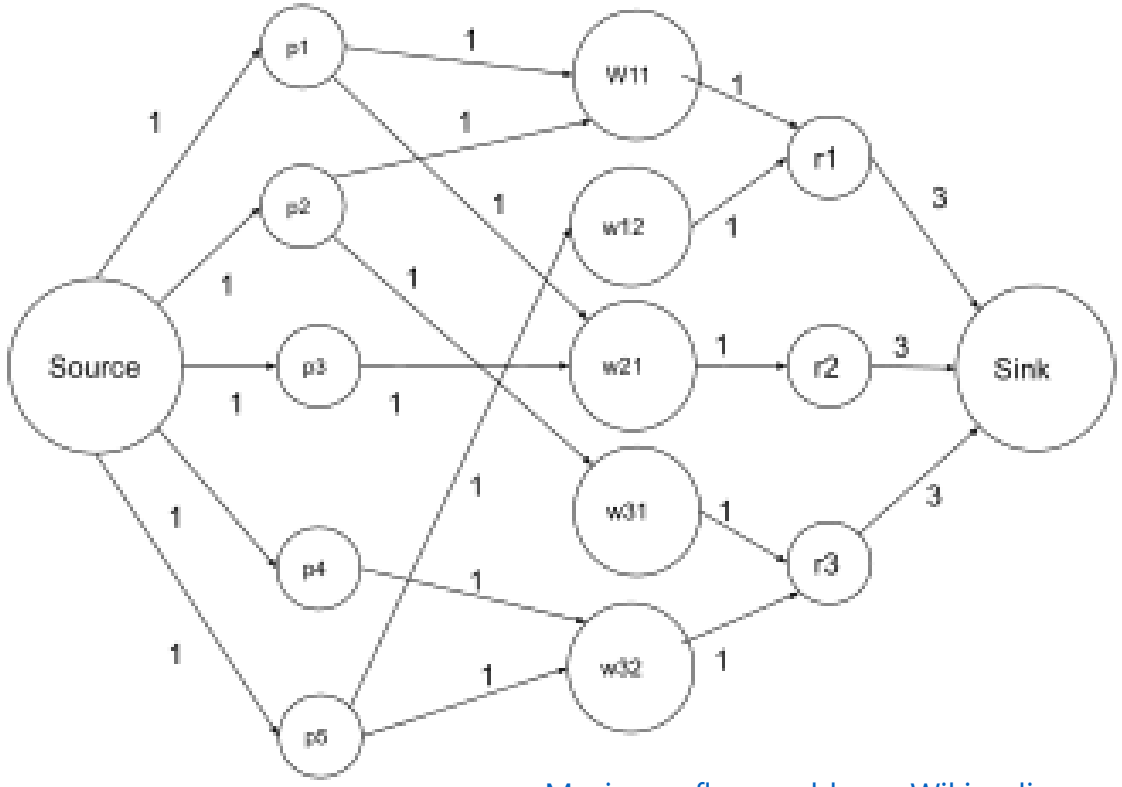
$$x_{ij} \geq 0 \ \ \forall (i,j) \in E$$

Max Flow

$$\max x_{ts}$$

$$\sum_{ij} x_{ij} - \sum_{ji} x_{ji} = 0 \; \forall \; i \in N$$

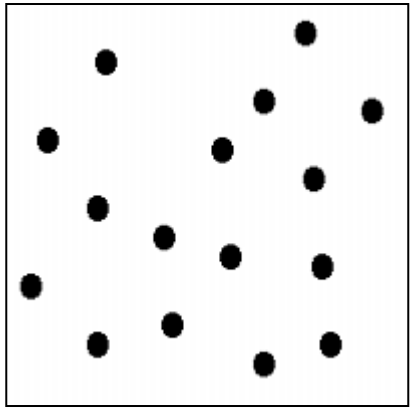
$$0 \leq x_{ij} \leq u_{ij} \; \forall \; i, j \in A$$

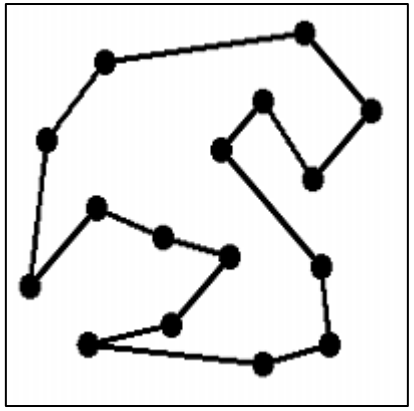


[Maximum flow problem - Wikipedia]

Traveling Salesman Problem (TSP)

Given n cities (nodes) and the cost of traveling, c_{ij}, from any city i to any other city j, find the minimum cost route that visits each city exactly once and then returns to the starting city.





MINIMUM SPANNING TREE PROBLEM

Given a set of nodes and the distances between them, connect these nodes with branches such as the total length of the branches is minimized and all nodes are connected.

Kruskal's Algorithm

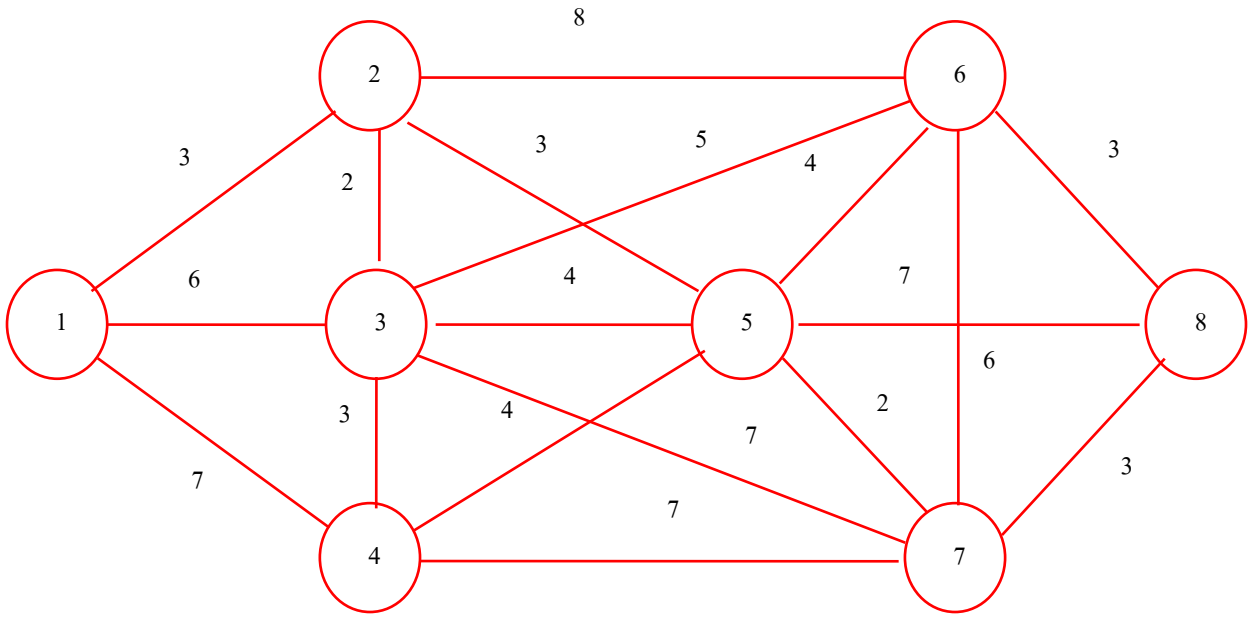
Step 1. Select any node arbitrarily and then connect it to the nearest node.

Step 2. Identify the unconnected node which is closest to any connected one and then connect these two nodes.

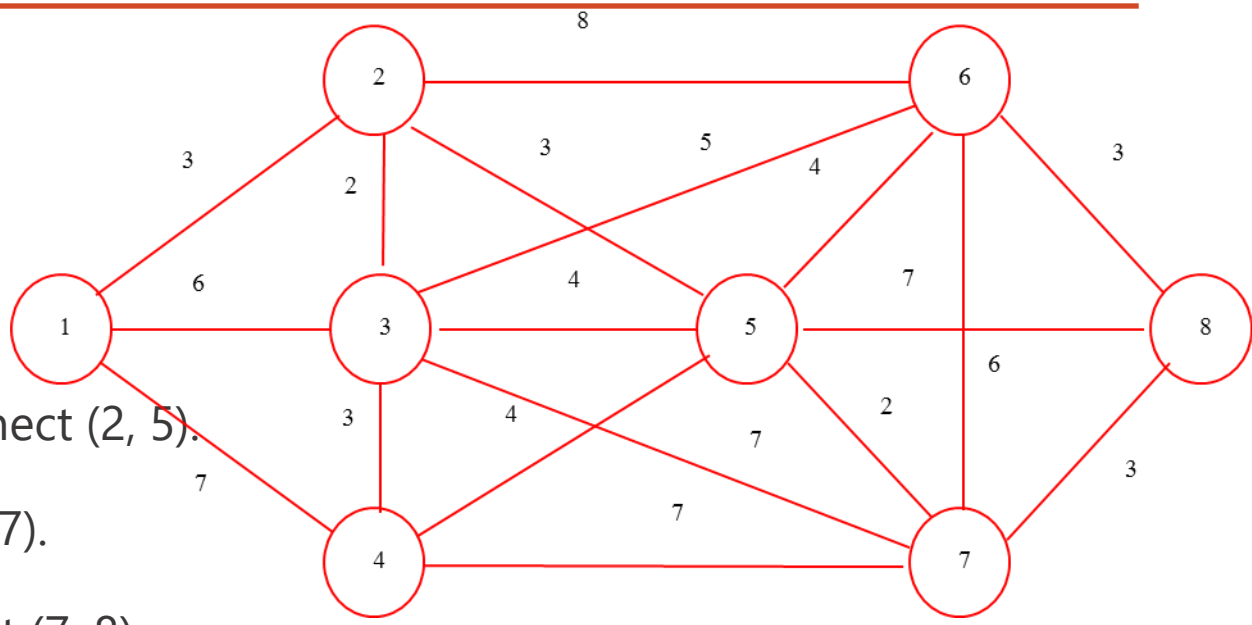
Step 3. If all nodes are connected, stop; the tree formed by the node connections is the minimum spanning tree. Otherwise, go to step 2.

Minimum Spanning Tree Example

Distances between 8 nodes to be connected are shown below. Find the minimum spanning tree.



Minimum spanning tree solution



Start with node 1.

Closest unconnected node: 2. Make connection (1, 2).

Closest unconnected node to (1, 2) is 3. Connect (2, 3).

Closest unconnected node to (1, 2, 3) is 5, or 4 (tie). Connect (2, 5).

Closest unconnected node to (1, 2, 3, 5) is 7. Connect (5, 7).

Closest unconnected node to (1, 2, 5, 7) is 8 or 4. Connect (7, 8).

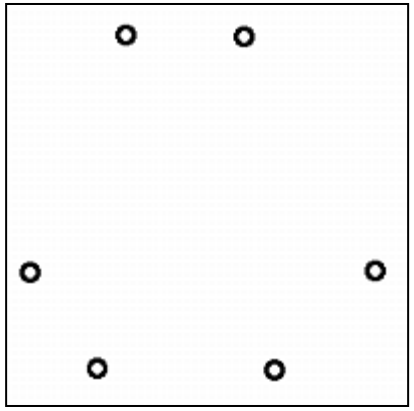
Closest unconnected node to (1, 2, 5, 7, 8) is 6 or 4 (tie). Connect (8, 6).

Closest unconnected node to (1, 2, 5, 7, 8, 6) is 4. Connect (3, 4).

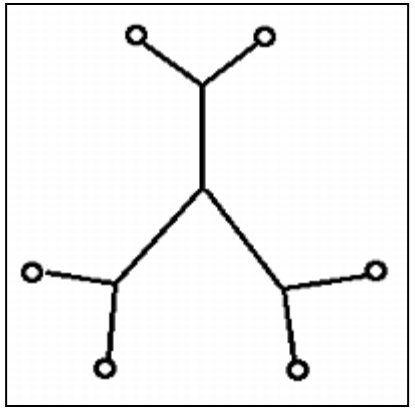
The ties were broken arbitrarily. But all the possible ways they are broken result in the same minimum spanning tree with length = 19.

Steiner Tree Problem

Given a set of nodes, connect the nodes at the minimum connection cost (distance). Intermediate connection nodes (junctions) and edges are allowed.



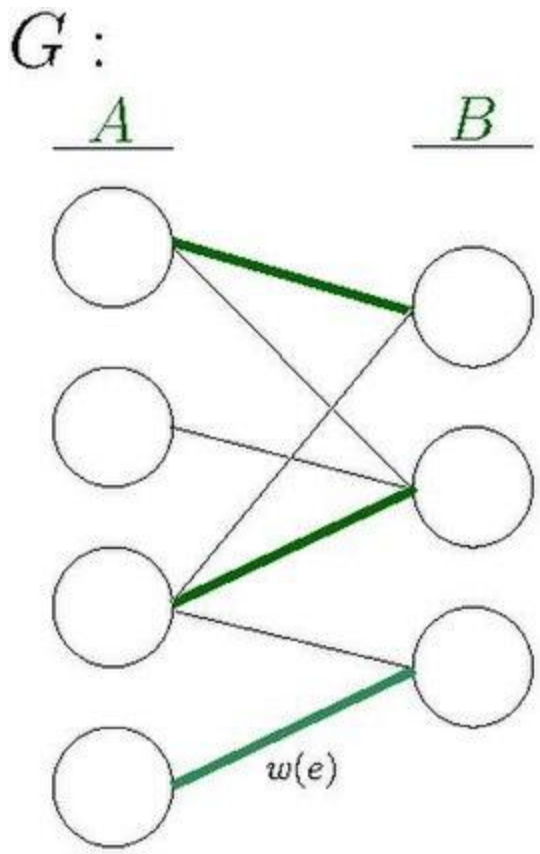
Input

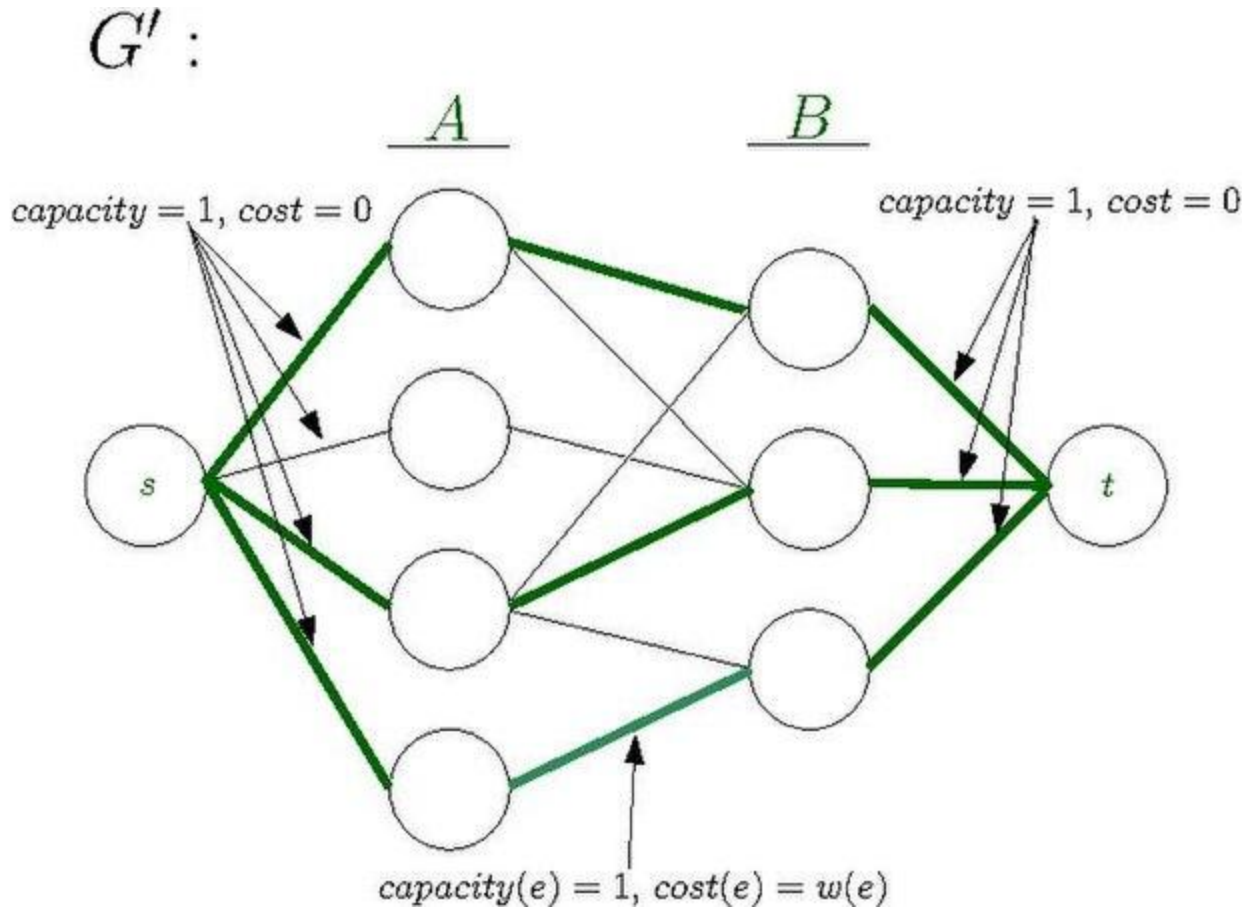


Output

The difference between this problem and the minimum spanning tree problem is that junctions can be used to reduce the cost (distance) of the tree.
Applications are in circuit layout and network design: VLSI design, designing networks of water pipes or heating ducts in buildings.

Transformations





[File:Minimum weight bipartite matching.pdf - Wikipedia]